

Claude Code Prompt Set

Cinematic, Nature-Meets-AI Portfolio — Phased Build Prompts

This document turns the Master Blueprint and Deep Prompt Bible into five ready-to-paste Claude Code prompts, one per build phase. Paste each phase's boxed prompt into Claude Code in order, review the output against the quality gate before moving on, and only then paste the next phase. Don't skip ahead — each phase assumes the previous one is done and approved.

Every phase carries the same project conventions: SESSION.md and ENGINEER_NOTES.md are gitignored from the first commit and never pushed; work happens in continuous single-pass sessions rather than being artificially paced across days; and git commits carry no AI co-author attribution. Phase 1 sets these up in a CLAUDE.md file so every later session picks them up automatically.

Source material: Claude_Code_Portfolio_Master_Blueprint.pdf, Claude_Code_Portfolio_Deep_Prompt_Bible.pdf

PHASE 1 — Research, Architecture & Design Specification

Paste first. No implementation code yet — planning, docs, and repo setup only.

Sets the creative and technical foundation: the mission, non-negotiable rules, narrative world structure, and confirmed build stack, then has Claude Code produce the full design spec, architecture, and roadmap before any scene is built — plus the CLAUDE.md conventions file every later phase relies on.

PROMPT — PHASE 1

You are helping me build a premium, interactive portfolio website. It must feel cinematic, spiritual, and unforgettable - never generic, cluttered, or like a junky template. Read this brief fully before doing anything.

MISSION

Build a cinematic, emotionally memorable portfolio that feels like an explorable spiritual journey rather than a website. The visitor should remember the experience before they remember the technology. Prioritize quality over quantity, calm over chaos, elegance over flashy effects. The site should feel like entering a living world - an enchanted forest where AI and nature coexist gently, not a flashy tech demo.

NON-NEGOTIABLE RULES

- No generic scroll animations, pop-ins, spinning cards, cheap bounce effects, chaotic scroll surprises, or random flying components.
- Every scene must have narrative meaning; every component must justify its existence. If a feature doesn't improve the visitor's understanding, memory, or emotional connection, cut it.
- Motion follows physics - spring, inertia, damping - never jitter or randomness.
- Respect prefers-reduced-motion; core content stays usable with effects off.
- Target 60 FPS, Lighthouse 95+, SEO-first, accessibility-first, even in the immersive mode.
- Research before deciding. Compare tools, read docs, inspect real examples, and choose the strongest option - don't default to the first idea. If a design or technical choice is uncertain, research it, compare alternatives, and recommend the best one with reasons.

NARRATIVE WORLD

The site is a journey through an enchanted forest, structured as story locations, not a scattered set of cards:

Entrance (gateway, first impression) -> Clearing/About (who I am, what drives me) -> Knowledge River (learning, iteration, growth) -> Animal Sanctuary (skills as symbolic creatures) -> Lab/Projects (projects as artifacts/chambers/machines) -> Observatory (achievements, certs, blog, metrics) -> Campfire/Contact (calm emotional close + contact links).

Earlier areas should be simple and calming; later areas can reveal more mechanical/AI complexity, so the site feels like progression, not randomness. Palette evolves with time of day (dawn/day/sunset/night) with smooth, gradual transitions - not toggle-switch jumps.

BUILD STACK (confirm or challenge with research)

Next.js + TypeScript, Tailwind, React Three Fiber + Three.js, GSAP, Framer Motion, Lenis (smooth scroll), Zustand (world state), Theatre.js, Howler.js (audio), shadcn/ui. If research turns up a stronger alternative for any of these, tell me and justify the swap before we commit to it.

YOUR TASKS FOR THIS PHASE (no implementation code yet)

1. Research the best implementation patterns for: smooth scroll + camera orchestration, R3F scene performance at 60fps, physics-based motion/easing, and time-of-day lighting systems. Tell me what you found and how it informs your choices.
2. Produce a complete design specification: visual direction, typography, color system per time-of-day, composition rules (foreground/midground/background, parallax, fog, focal points).
3. Produce a file/folder architecture for the whole project (immersive + classic + shared content layer).
4. Produce a scene graph covering all seven story locations and how they connect.
5. Produce state machine diagrams (clear markdown/mermaid) for: world time-of-day state, navigation/scene state, and interaction/progress state (what's been viewed/unlocked).
6. Produce an asset list (3D models, textures, audio, fonts, icons) with intended sources (Poly Haven, Kenney, Sketchfab-licensed, Google Fonts, Lucide, LottieFiles, SVG Repo, etc.) and license notes for each.
7. Produce an animation bible: for each motion type (camera, scroll, hover, scene transition, object motion, theme transition, UI motion) specify the required feel and technique.
8. Produce an accessibility checklist and a testing plan (unit/integration/visual/perf) for the whole build.
9. Produce a phased implementation roadmap for the remaining phases (Engine & Motion -> Story Scenes -> Features & Classic Mode -> Optimization/SEO/Launch), with deliverables and acceptance criteria per phase.
10. Set up the repository: initialize the project with the confirmed stack, add a .gitignore that excludes SESSION.md and ENGINEER_NOTES.md before the first commit, and create CLAUDE.md capturing the conventions below plus everything you decide in this phase.

PROJECT CONVENTIONS TO ENCODE IN CLAUDE.md

- Before the first commit, add SESSION.md and ENGINEER_NOTES.md to .gitignore. These are private working files and must never be committed or pushed.
- Maintain SESSION.md as a running log of what you built, decided, and deferred; maintain ENGINEER_NOTES.md as a technical scratchpad (open questions, TODOs, gotchas). Update both continuously as you work, not just at the end.
- Build in continuous single-pass sessions. Do not artificially pace work across days or weeks. Work through each phase's deliverables in one continuous pass unless genuinely blocked and needing my input.
- Do not add AI co-author attribution to git commits (no "Co-Authored-By: Claude" trailer or similar). Commit messages should read as written by the engineer alone.

Don't write feature code yet - this phase ends with the plan, the diagrams, the asset/animation/testing docs, and a clean initialized repo with conventions in place. Flag anything in this brief you think is weak and propose a stronger alternative with your reasoning.

PHASE 2 — Engine, World Foundation & Motion System

Paste after Phase 1's plan is reviewed and approved.

Builds the systems every scene will sit on: world-state, time/weather, camera + scroll, scene transitions, audio, and the performance governor — proven with one minimal test scene rather than final content.

PROMPT — PHASE 2

Continue following CLAUDE.md conventions (gitignored SESSION.md/ENGINEER_NOTES.md, continuous single-pass building, no AI co-author attribution in commits).

Build the underlying engine that every scene will run on top of - no final story scenes yet, just the systems and a minimal test scene to prove they work together.

BUILD:

1. Core app shell - Next.js routing for the immersive route and a placeholder /classic route (fleshed out in Phase 4), shared layout, design tokens from Phase 1's spec wired into Tailwind config.
2. World-state engine (Zustand) - single source of truth for: current time-of-day, current scene/chapter, camera state, audio state, interaction/progress state (what's been viewed/unlocked), and a device-capability/performance tier.
3. Time system - drives lighting, sky color, shadow behavior, fog density, and which animals/behaviors are active, synced to local time. Transitions between dawn/day/golden-hour/night must be gradual (interpolated over minutes), never an instant swap. Optional weather layer (mist/rain/breeze/light snow) that composes with time-of-day rather than fighting it.
4. Camera & scroll system - Lenis-driven custom scroll mapped to journey progress through the world; camera movement uses inertia/damping/gentle acceleration so it feels guided, not dragged. No jarring snap points.
5. Scene-transition system - crossfades, light sweeps, fog movement, or cinematic camera moves between chapters, built as a reusable orchestration layer (Theatre.js and/or GSAP timelines), not copy-pasted per scene.
6. Audio system (Howler.js) - layered ambient loops, scene-specific SFX, hover/confirmation sounds, a global mute, and a volume control, all keyed off world-state so audio shifts with time/scene automatically.
7. Performance governor - detects device capability and scales particle counts, shadow quality, and 3D detail accordingly; wire in a reduced-motion mode that disables non-essential animation and honors prefers-reduced-motion.
8. Build ONE minimal test/proof scene (simple geometry is fine) that exercises every system above - time-of-day lighting change, scroll-driven camera move, a transition, ambient audio, and the performance governor scaling something - so we can validate the engine before building all seven real scenes.

For this phase, give me: a research summary of the patterns you used (scroll orchestration, camera easing, time-of-day lighting) and why, the implementation plan, suggested improvements over the Phase 1 spec if you found a better approach mid-build, risk notes (anything fragile or likely to need rework once real scenes land), and the actual code. Update SESSION.md/ENGINEER_NOTES.md as you go (gitignored, not committed).

PHASE 3 — Story World — All Scenes

Paste once the engine from Phase 2 is validated and approved.

Builds all seven story locations on the Phase 2 engine, in narrative order, each treated as a chapter with its own meaning, depth, and restraint — not a generic page section.

PROMPT — PHASE 3

Continue following CLAUDE.md conventions.

Build all seven story locations on top of the Phase 2 engine, one at a time, in narrative order - each is a chapter, not a generic "section":

1. Entrance / Hero Gateway - first impression: title, atmosphere, a clear, calm invitation to begin the journey.
2. Clearing / About - who I am, what drives me, the themes that shape my work.
3. Knowledge River - a flowing path motif representing learning, research, iteration, growth.
4. Animal Sanctuary / Skill Sanctuary - skills represented as symbolic/mechanical creatures, one per domain, with believable idle states and movement paths (wildlife choreography) rather than static poses.
5. Lab / Project Chamber - projects presented as artifacts, discoveries, or machines rather than plain cards; each project supports a deep-dive view (architecture, outcome, visuals) and a focused "case study" mode surfacing impact metrics for recruiters.
6. Observatory - achievements, certifications (as a ceremonial timeline/wall with date, issuer, significance), and public proof of work.
7. Campfire / Contact - a calm, emotionally memorable close: contact links, social links, resume export.

For every scene:

- Use the Phase 2 world-state, camera/scroll, transition, and audio systems - don't build one-off logic per scene.
- Compose foreground/midground/background layers with parallax and fog for depth; keep it restrained - quiet moments are part of the design, not everything needs to move at once.
- Add a small number of hidden lore/narrative objects across the world (subtle, optional discovery, not required for comprehension) and at least one easter egg beyond text (e.g. Konami code triggering a hidden cave/creature or an alternate atmosphere).
- Add a dynamic skybox tied to the Phase 2 time system so each scene reads correctly at any time of day.
- Ensure every scene degrades gracefully: content stays legible and navigable if WebGL/advanced effects fail, and on tablet/mobile viewports.
- Document and add basic tests for each scene as you finish it (build each subsystem independently with documentation and tests) before moving to the next.

Work through all seven in one continuous pass. After each scene, briefly self-check against the quality gate: does this feel premium and calm, does it feel like a story chapter rather than a template section, would it still work with the fancy layers stripped? Fix before moving on rather than batching fixes at the end.

For this phase, give me: research notes for any scene-specific pattern you needed (e.g. instancing for the sanctuary's creatures, project deep-dive layout patterns), the implementation plan, suggested improvements, risk notes, and the code for all seven scenes.

PHASE 4 — Features, Integrations & Classic Mode

Paste once all seven scenes are built and approved.

Layers in the supporting features with restraint, and builds the classic lightweight alternative experience as a genuine second route, not an afterthought.

PROMPT — PHASE 4

Continue following CLAUDE.md conventions.

Two goals this phase: (A) layer in the supporting features with restraint, and (B) build the classic lightweight alternative experience.

A) FEATURES (each must clear the bar: does it improve understanding, memory, or emotional connection - if not, cut or simplify it)

- Smooth dark/light (day/night) mode as a gradual atmosphere shift (sky, shadow, fog, particles, sound together), not a flat color-token swap.
- Optional ambient soundtrack with layered SFX (wind, water, birds, distant sounds), safe mute + volume control (should already be wired from Phase 2 - surface the UI here).
- Voice assistant for natural navigation (e.g. "show my AI projects", "open the research area") that maps intents to the existing scene/navigation state.
- Embedded AI chatbot grounded in my resume, projects, and portfolio structure - able to answer questions about my background and jump the visitor to relevant scenes, and support in-portfolio search.
- Live GitHub activity (contributions/commit highlights), a compact LeetCode stats display, and a latest-blog-posts area presented as a living archive, not a noisy feed.
- Certifications timeline (already built as the Observatory wall in Phase 3 - wire real data in here).
- Animated skill tree as an organic tree/constellation/root system that lights up by domain (ties into the Sanctuary scene).
- Keyboard shortcuts (P, R, G, C, /) with visible hints and an accessible help overlay.
- Scene bookmarks so a visitor can jump to a chapter without breaking the story feel.
- Resume export (clean PDF) reachable from both classic and immersive views.
- A tasteful, fast preloader that prepares the world without wasting the visitor's time.
- For any external asset (Motion Primitives, Haikei, Spline, Blender assets, Poly Haven, Kenney, Sketchfab, LottieFiles, SVG Repo, shader references, Aceternity UI, Magic UI): verify license compatibility and optimize before inclusion, and note the source in ENGINEER_NOTES.md.

B) CLASSIC MODE

- Build /classic as a genuinely polished second experience, not an afterthought: semantic HTML/CSS, minimal JS, same copy and structure as the immersive site, stripped-down visuals, fast by default.
- Add a visible, easy toggle between Classic and Immersive (consider a landing choice on first visit) that preserves the visitor's place in the content.
- Keep content consistent across both modes so the portfolio reads as one coherent story either way.

For this phase, give me: a research summary for anything non-trivial (voice intent handling, chatbot grounding approach, license checks on sourced assets), the implementation plan, suggested improvements, risk notes, and the code for both the features and classic mode.

PHASE 5 — Optimization, Accessibility, SEO & Launch QA

Paste last, once everything above is built and functionally complete.

No new scenes or features here — this phase makes everything fast, discoverable, accessible, and launch-ready, then runs the final quality gate honestly.

PROMPT — PHASE 5

Continue following CLAUDE.md conventions.

This phase doesn't add new scenes or features - it makes everything fast, discoverable, accessible, and launch-ready.

PERFORMANCE

- GPU-friendly rendering: instancing for repeated objects (creatures, foliage, particles), LOD where it matters, lazy loading and code splitting for anything not needed at first paint.
- Optimize textures to KTX2 and compress models to glTF/Draco; prefetch only truly critical assets.
- Confirm the reduced-motion mode and performance governor (from Phase 2) actually degrade the experience gracefully on lower-end devices - test, don't assume.
- Eliminate layout shift; target 60 FPS in the immersive mode and Lighthouse 95+ in a realistic production build for both routes.

SEO

- Semantic HTML throughout (including inside the immersive shell), full metadata, Open Graph tags, dynamic OG images, JSON-LD Person schema, sitemap.xml, robots.txt, canonical URLs, descriptive image alt text, and a properly structured blog.
- Prefer server-rendering or static generation wherever it helps performance or discoverability without hurting the experience.

ACCESSIBILITY

- Full keyboard navigation (including the shortcut system and immersive scene navigation), visible focus states, alt text everywhere, prefers-reduced-motion fully respected, contrast-safe palette across all time-of-day states, and readable fallbacks if 3D/WebGL isn't available.
- Run this checklist end-to-end and fix anything that fails rather than noting it as future work.

FINAL QUALITY GATE - answer honestly for the finished site, and fix/simplify/rework before calling it done if any answer is "no":

- Would a recruiter remember this site after leaving it?
- Does the motion feel premium and calm, never jittery or excessive?
- Does it feel like a story rather than a UI template?
- Does it still work with the fancy layers removed (reduced motion / WebGL failure)?
- Is the classic version polished enough to stand alone?
- Does it load fast enough to feel serious and professional?

Finish with a release checklist (what's verified, what's not, any known follow-ups) and a final SESSION.md/ENGINEER_NOTES.md update summarizing the whole build. For this phase, give me: the audit results (performance/SEO/accessibility, with numbers where possible), what you fixed, what still needs my input, and the final code changes.